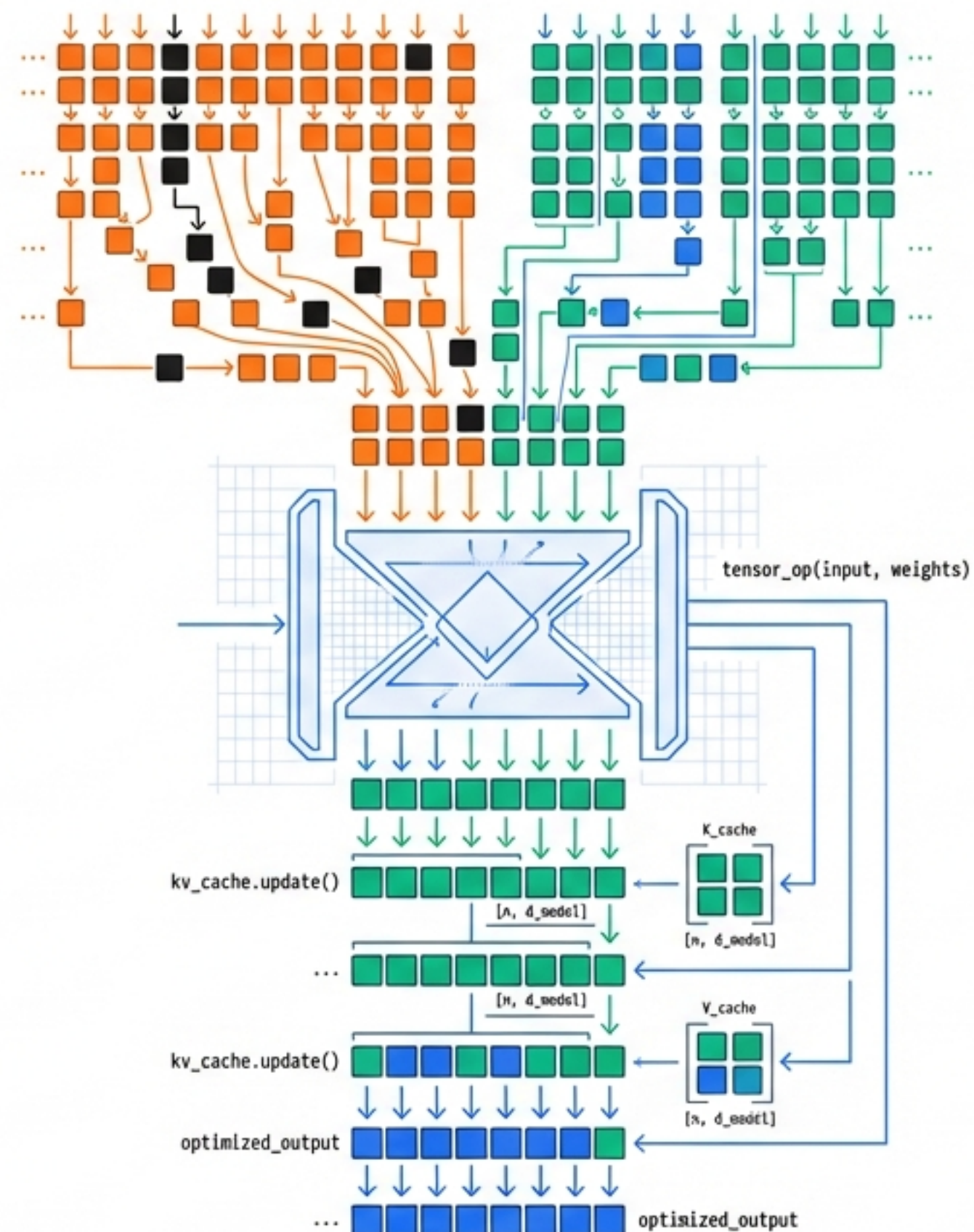


เจาะลึกและเขียนโค้ด KV Cache ใน LLMs ตั้งแต่ศูนย์

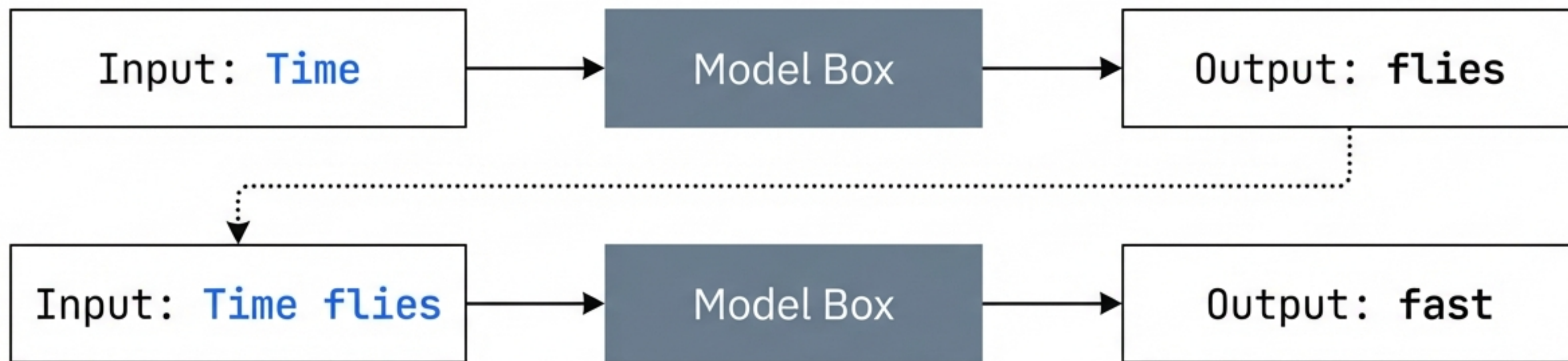
เทคนิคสำคัญเบื้องหลังการเร่งความเร็ว
Inference ของ ChatGPT และ Llama

เรียบเรียงจากบทความของ Sebastian Raschka, PhD



LLM สร้างข้อความอย่างไร?

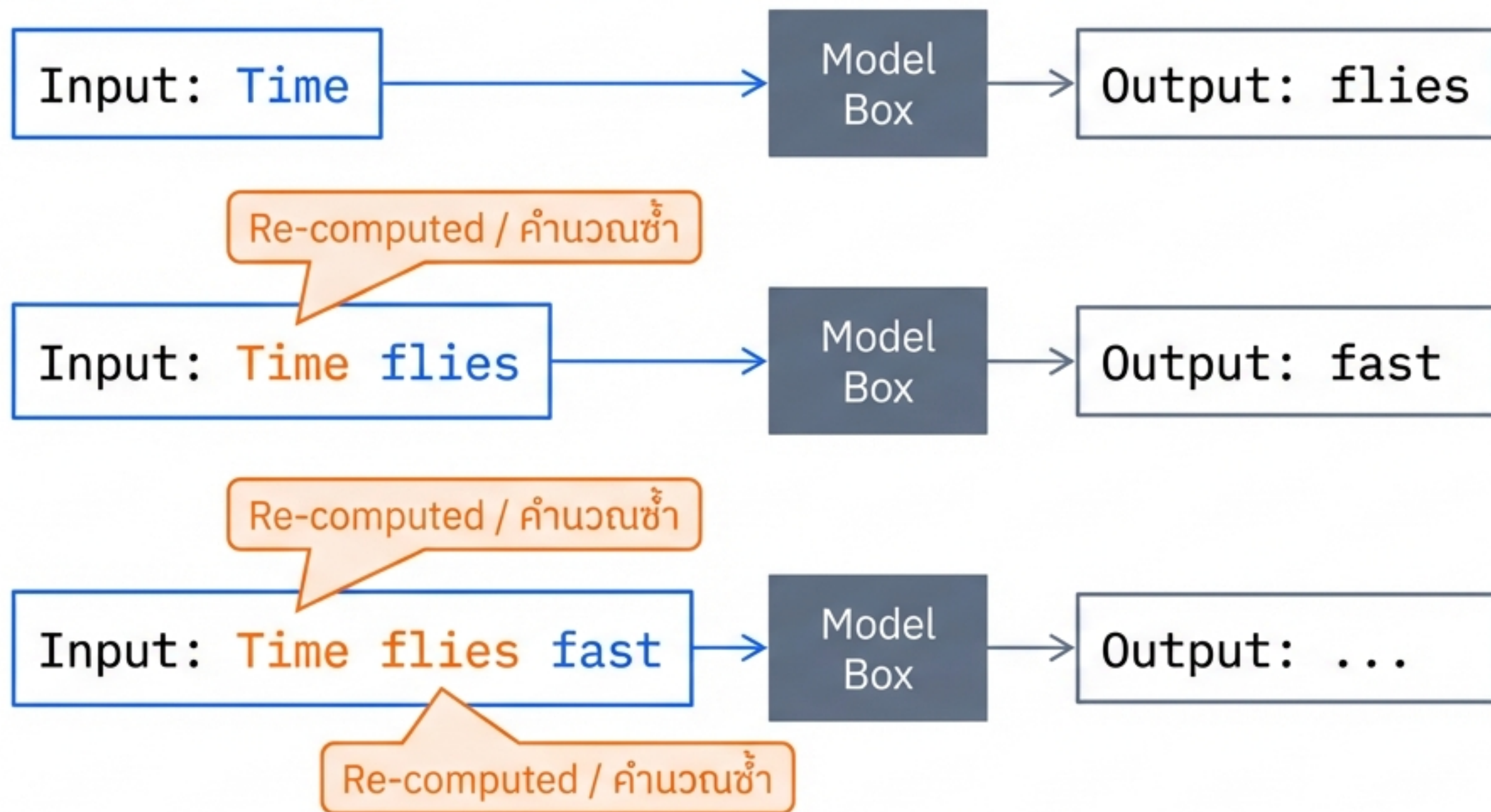
Autoregressive Process



โมเดลประมวลผลแบบ Autoregressive: สร้างทีละ Token โดยใช้บริบทก่อนหน้าทั้งหมด

ความไร้ประสิทธิภาพที่ซ่อนอยู่

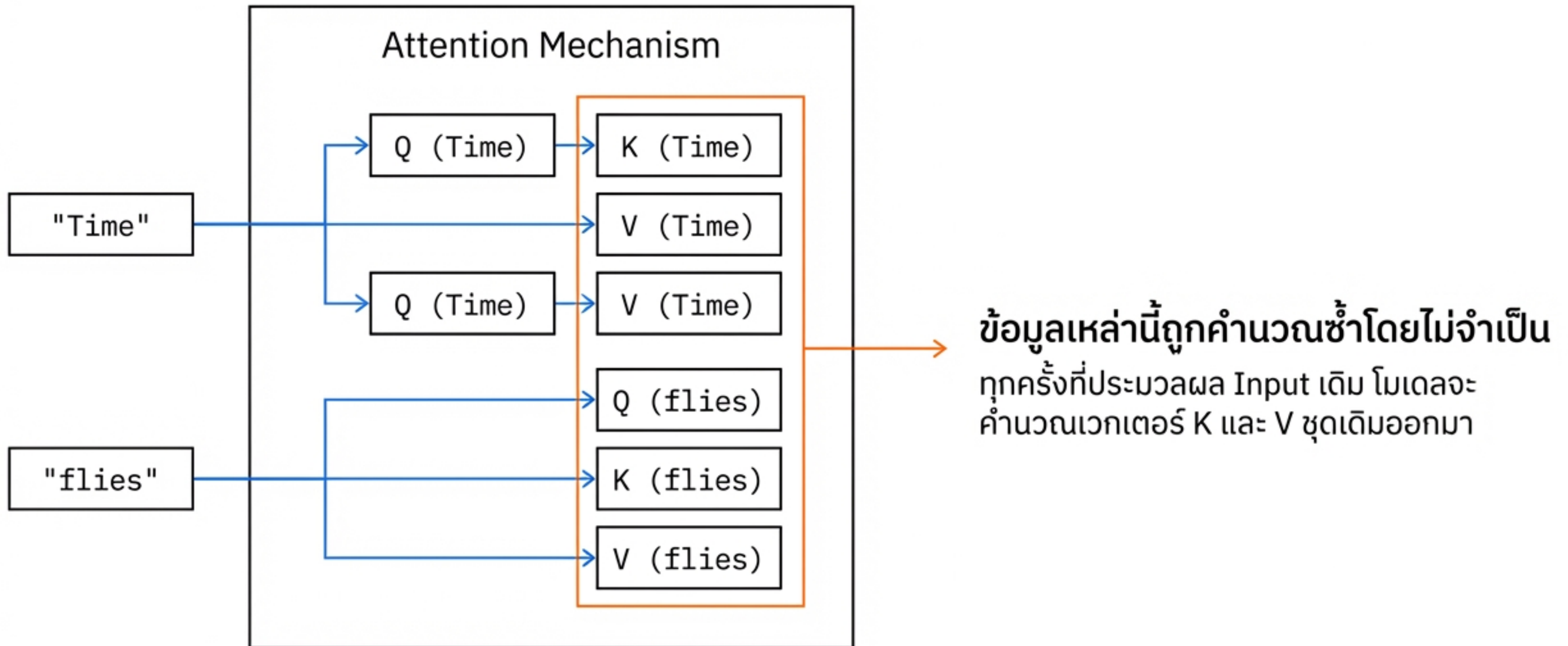
The Redundancy Problem



เมื่อสร้างคำว่า 'fast'
โมเดลต้องคำนวณ 'Time'
และ 'flies' ซ้ำอีกรอบ

ยิ่งประโยคยาว
การคำนวณซ้ำยิ่งทวีคูณ
(Complexity $O(n^2)$)

คอบวอดอยู่ที่ Attention Mechanism



ทางออก: KV Cache คืออะไร?

กลไกการ 'แคช' (เก็บ) เวกเตอร์ K และ V ที่คำนวณเสร็จแล้วไว้ใช้ในรอบถัดไป แทนที่จะคำนวณใหม่

Without Cache

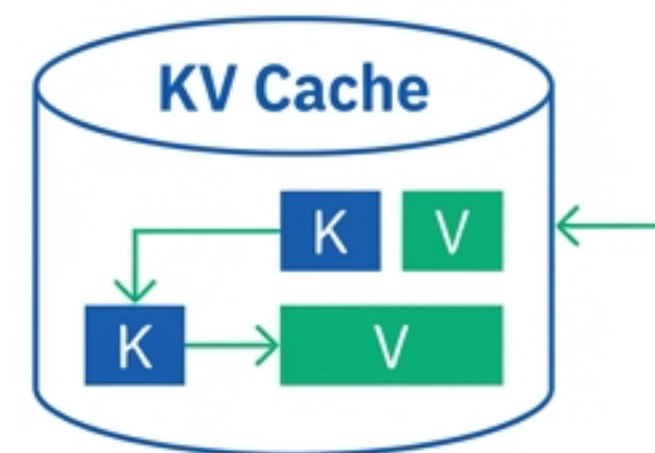
$\text{Calc}(A+B) \rightarrow \text{Calc}(A+B+C) \rightarrow \text{Calc}(A+B+C+D)$

With Cache

$\text{Calc}(A) \rightarrow \text{Store}(A)$

$\text{Calc}(B) + \text{Retrieve}(A)$

$\text{Calc}(C) + \text{Retrieve}(A, B)$



เปรียบเทียบการทำงาน (Without vs With Cache)

No Cache - $O(n^2)$

Step 1

[Time]

Step 2

[Time][flies]

Step 3

[Time][flies][fast]

With KV Cache - $O(n)$

[Time]

Save to Cache

KV Cache

[flies] + (Retrieve [Time])

[fast] + (Retrieve [Time][flies])

‘Time’ ถูกคำนวณแค่ครั้งเดียว และนำกลับมาใช้ซ้ำตลอดการสนทนา
ลด Complexity จาก Quadratic $O(n^2)$ เป็น Linear $O(n)$

จากแนวคิดสู่การเขียนโค้ด



- การแปลงทฤษฎีให้เป็น Python (PyTorch) โดยแก้ไขโค้ด LLM ดั้งเดิม
- เราจะโฟกัสที่การเปลี่ยนแปลง 3 จุดหลักในคลาส 'MultiHeadAttention' และ 'GPTModel'

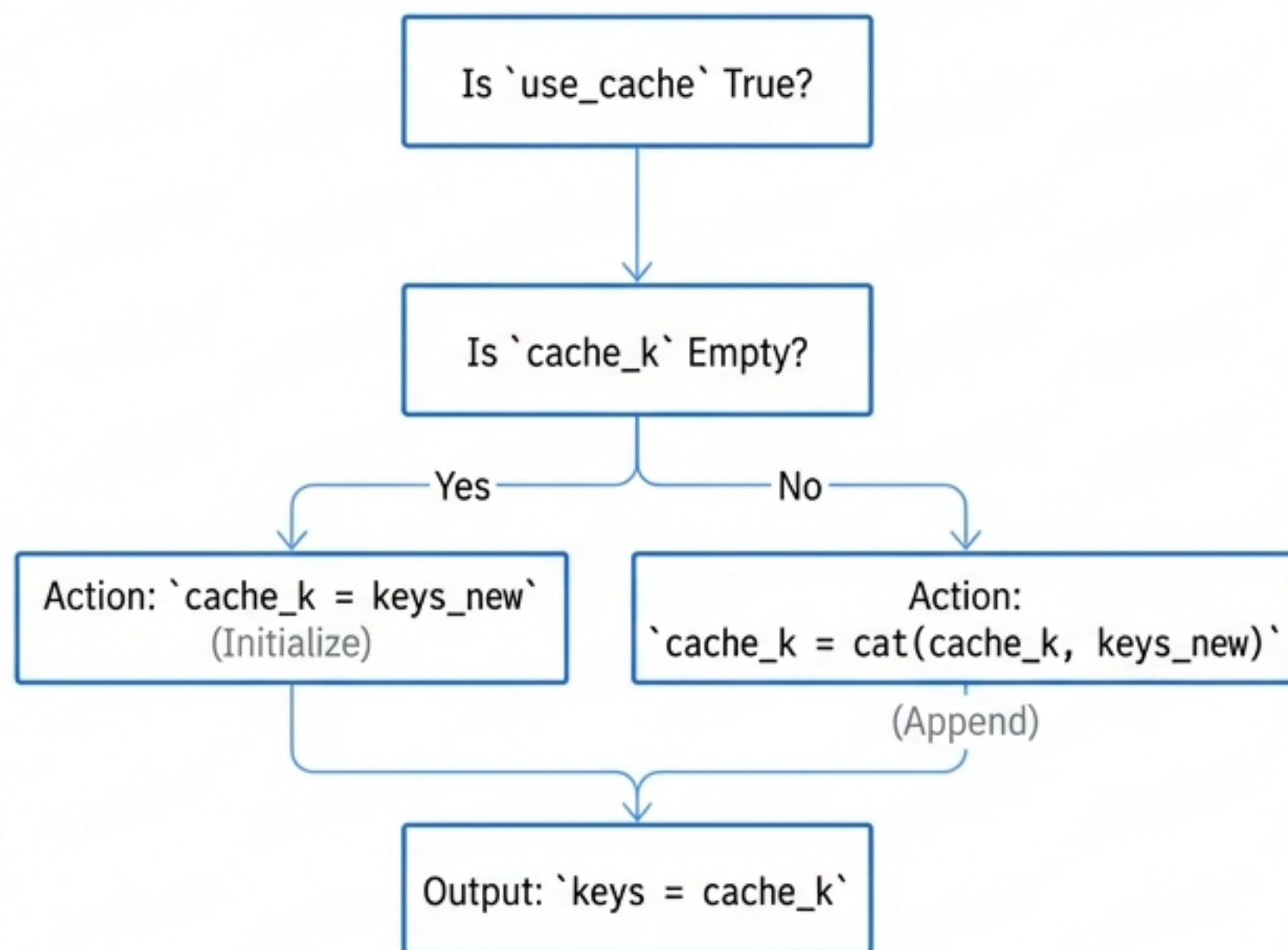
Step 1: เตรียมพื้นที่เก็บข้อมูล (Register Buffers)

```
class MultiHeadAttention(nn.Module):  
    def __init__(self, d_in, d_out, context_length, ...):  
        super().__init__()  
        # ... other inits ...  
  
        # NEW: Register buffers for Cache  
        self.register_buffer("cache_k", None)  
        self.register_buffer("cache_v", None)
```

ใช้ Buffer เพราะต้องการเก็บ State ระหว่างการรันแต่ไม่ต้องการ Gradient (Not a trainable parameter)

เริ่มต้นด้วยค่าว่าง

Step 2: หัวใจสำคัญ (Store & Retrieve)



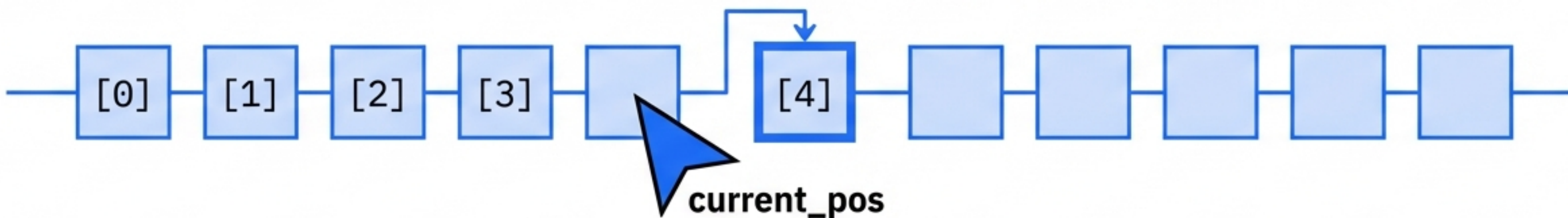
```
def forward(self, x, use_cache=False):  
    # ... calculation of keys_new, values_new ...  
  
    if use_cache:  
        if self.cache_k is None:  
            self.cache_k = keys_new  
            self.cache_v = values_new  
        else:  
            self.cache_k = torch.cat([self.cache_k,  
                                     keys_new], dim=1)  
            self.cache_v = torch.cat([self.cache_v,  
                                     values_new], dim=1)  
  
    keys, values = self.cache_k, self.cache_v
```

Step 3: การจัดการตำแหน่ง (Position Tracking)

ปัญหา: เมื่อส่งเข้าโมเดลทีละ Token โมเดลจะนึกว่าอยู่ที่ตำแหน่ง 0 เสมอ ถ้าไม่บอก `current\_pos`

```
# Inside GPTModel forward
if use_cache:
    # Generate position IDs starting from current_pos
    pos_ids = torch.arange(self.current_pos, self.current_pos + seq_len, ...)

    # Update cursor
    self.current_pos += seq_len
else:
    pos_ids = torch.arange(0, seq_len, ...)
```



ตัวแปร `current\_pos` ช่วยให้ Token ใหม่ 'ต่อแถว' ได้ถูกต้อง

การใช้งานจริงใน Loop (The Inference Loop)

```
# Initial Context Processing
logits = model(idx[:, -ctx_len:], use_cache=True)

# Generation Loop
for _ in range(max_new_tokens):
    # Get the last predicted token
    next_idx = logits[:, -1].argmax(dim=-1, keepdim=True)

    # CRITICAL: Feed ONLY the new token into the model
    logits = model(next_idx, use_cache=True)

    idx = torch.cat((idx, next_idx), dim=1)
```

- Without Cache:
model(idx) ->
ส่งทั้งประโยค ($O(n^2)$)
- With Cache:
model(next_idx) ->
ส่งแค่คำล่าสุด ($O(n)$)

ผลลัพธ์: เร็วขึ้นจริงไหม?

Benchmark on M4 Chip (CPU) – 124M Parameter Model



- เร็วขึ้นประมาณ 5 เท่า สำหรับการเจน 200 Tokens
- ผลลัพธ์ข้อความ (Output Text) เหมือนเดิม 100% ยืนยันความถูกต้องของ Algorithm

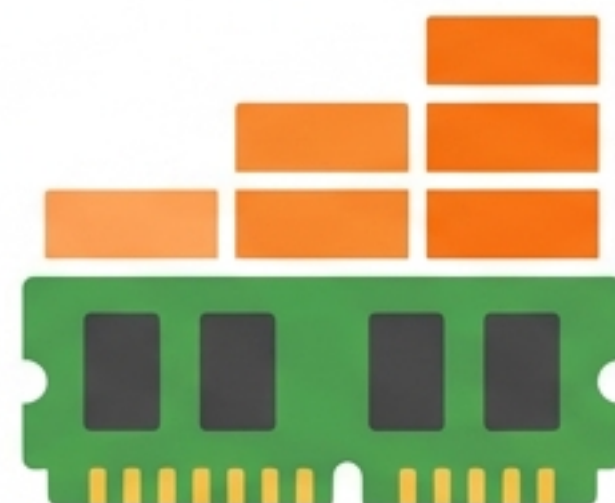
ดาบสองคม: ข้อดี vs ข้อเสีย

Computational Efficiency



- Complexity ลดลงเหลือ $O(n)$
- ประมวลผลเร็วขึ้นมาก ลด Latency

Memory Usage

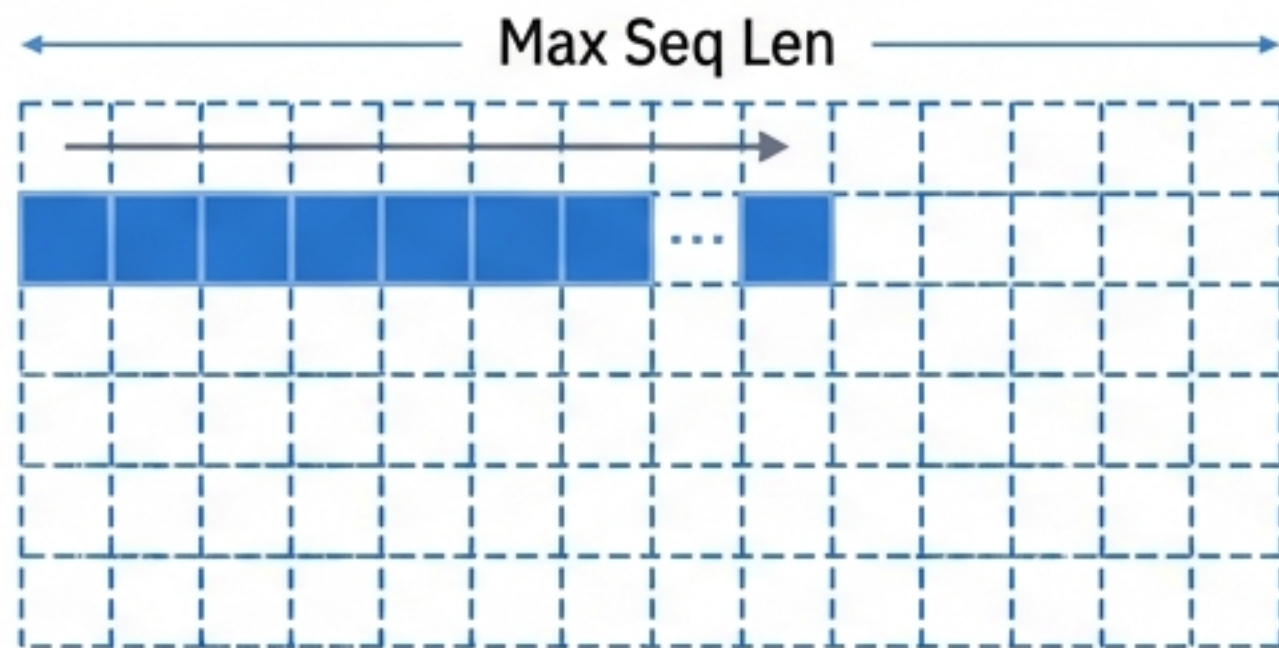


- กิน RAM เพิ่มขึ้นแบบ Linear ตามความยาวประโยค (Context Length)
- สำหรับ Context ยาว (128k tokens) Cache อาจกินพื้นที่ GPU มหาศาล

การปรับปรุงขั้นสูง (Production Optimization)

`torch.cat` สร้าง Memory Fragmentation และช้าเมื่อทำซ้ำๆ

Pre-allocation



จองพื้นที่เมมโมรีขนาดใหญ่ไว้ล่วงหน้า (Max Seq Len)
แล้วค่อยๆ เขียนข้อมูลลงไป (Zero-copy)

Sliding Window



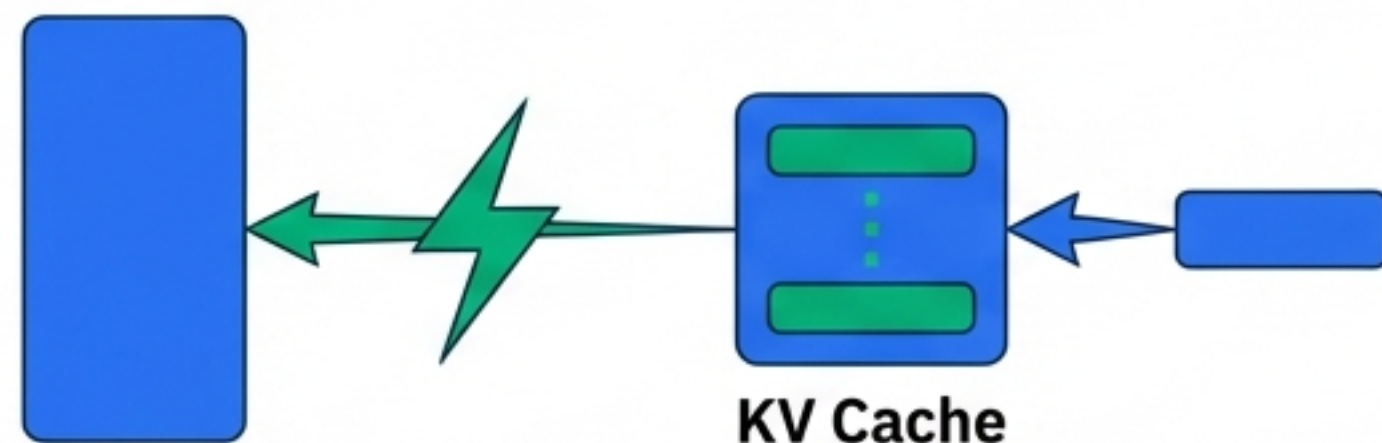
เก็บรักษา Cache เฉพาะ n คำล่าสุด
(Truncation) เพื่อจำกัดการใช้ RAM

โมเดลอย่าง Llama 3 หรือ Qwen ที่มี Context ยาวมาก ใช้เทคนิคเหล่านี้ร่วมกับ `torch.compile`

บทสรุป (Key Takeaways)

- **Speed:** KV Cache เป็นสิ่งจำเป็นสำหรับ Production Inference (เร็วขึ้น 5x+ ใน
- **Logic:** หลักการคือ "จำแล้วไม่คิดใหม่" (Compute once, reuse often).
- **Future:** การจัดการ Memory คือความท้าทายถัดไป (PagedAttention, vLLM)

Input -> Cache -> **Lightning Fast Output**



ดาวน์โหลดและทดลองรันโค้ดจริงได้ที่
GitHub: [rasbt/LLMs-from-scratch](https://github.com/rasbt/LLMs-from-scratch)